VoltEZ

Complete API Reference & System Architecture

FastAPI Backend | Flutter Frontend | ML Models | WebSocket

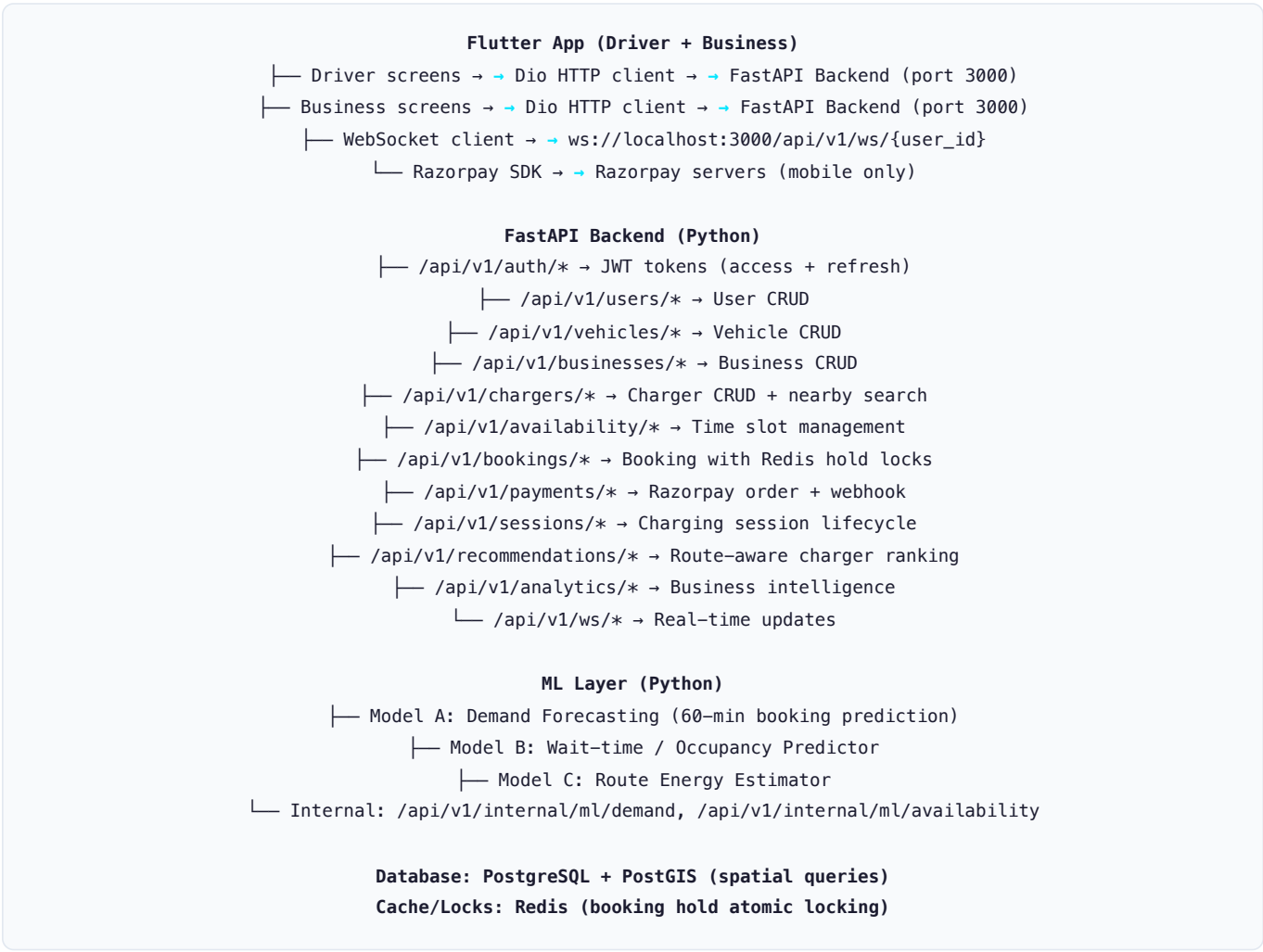
Generated: August 2026

Branch: `final-frontend` | Backend: `backend-kalpesh` | ML: `ML-Final`

Backend Base URL: `http://localhost:3000/api/v1`

Table of Contents

01 — System Architecture Overview



02 — Authentication & Authorization

METHOD	ENDPOINT	AUTH	REQUEST BODY	RESPONSE
POST	/auth/register	Public	{ name, email, password, role: "DRIVER"|"OWNER", phone? }	UserResponse (201)
POST	/auth/login	Public	OAuth2 form data: username={email}, password={password}	{ access_token, refresh_token, token_type: "bearer" }
POST	/auth/refresh	Refresh Token	{ refresh_token }	TokenResponse (new tokens)

JWT Token Flow

```
Register/Login → Receive access_token (15min) + refresh_token (7 days)
→ All subsequent requests include Authorization: Bearer <access_token>
→ On 401: Frontend automatically calls /auth/refresh with refresh_token
     → If refresh fails: User is logged out, redirected to /login
```

User Roles

ROLE	DESCRIPTION	CAN DO
DRIVER	EV driver who books and charges	Register vehicle, search chargers, book, charge, rate
OWNER	Charger station business owner	Manage business, chargers, availability, view analytics
ADMIN	Platform administrator	Full access to all resources

03 — User Management

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
GET	/users/me	Bearer	—	UserResponse
PATCH	/users/me	Bearer	{ name?, phone? }	UserResponse

UserResponse Schema

```
{
  "id": 1,                    // int (backend primary key)
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "+919876543210",   // nullable
  "role": "DRIVER",           // "DRIVER" | "OWNER" | "ADMIN"
  "verification_status": "unverified",
  "created_at": "2026-08-18T10:00:00Z"
}
```

04 — Vehicle Management

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
GET	/vehicles	Driver	—	List<VehicleResponse>
POST	/vehicles	Driver	VehicleCreate	VehicleResponse (201)
GET	/vehicles/{id}	Driver	—	VehicleResponse
PATCH	/vehicles/{id}	Driver	VehicleUpdate	VehicleResponse
DELETE	/vehicles/{id}	Driver	—	204 No Content

VehicleCreate Schema

```
{
  "make": "Tata",                    // required
  "model": "Nexon EV",               // required
  "battery_kwh": 40.5,               // required, > 0
  "connector_types": ["CCS2"],        // required, min 1 item
  "max_ac_kw": 7.2,                   // optional
  "max_dc_kw": 60.0,                  // optional
  "estimated_range_km": 312.0         // optional
}
```

05 — Business (Charger Owner) Management

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
GET	/businesses	Owner	—	List<BusinessResponse>
POST	/businesses	Owner	BusinessCreate	BusinessResponse (201)
GET	/businesses/{id}	Bearer	—	BusinessResponse
PATCH	/businesses/{id}	Owner	BusinessUpdate	BusinessResponse
DELETE	/businesses/{id}	Owner	—	204 No Content

BusinessResponse Schema

```
{
  "id": 1,
  "owner_id": 1,
  "name": "ABC Motors",
  "category": "mall",                    // "mall" | "office" | "apartment"
  "address": "123 MG Road, Bangalore",
  "opening_hours": {
    "mon": {"open": "09:00", "close": "21:00"},
    "tue": {"open": "09:00", "close": "21:00"}
  },
  "verification_status": "verified",
  "latitude": 12.9716,
  "longitude": 77.5946,
  "created_at": "2026-08-18T10:00:00Z",
  "updated_at": "2026-08-18T10:00:00Z"
}
```

06 — Charger & Port Management

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
POST	/chargers	Owner	ChargerCreate	ChargerResponse (201)
GET	/chargers/nearby	Public	?latitude=...&longitude=...&radius_meters=5000	List<ChargerResponse>
GET	/chargers/{id}	Bearer	—	ChargerResponse
POST	/chargers/{id}/report-issue	Driver	—	204 No Content

ChargerCreate Schema

```
{
  "business_id": 1,                  // required
  "name": "Phoenix Mall Charger",    // required
  "power_kw": 60.0,                  // required, > 0
  "access_type": "public",           // "public" | "private" | "restricted"
  "base_price": 14.0,                // required, INR per kWh
  "status": "active",                // "active" | "paused" | "inactive"
  "parking_info": "B2 Near Elevator",// optional
  "amenities": "WiFi, Food",         // optional, comma-separated
  "latitude": 12.9716,               // required
  "longitude": 77.5946               // required
}
```

ChargerResponse Schema

```
{
  "id": 1,
  "business_id": 1,
  "name": "Phoenix Mall Charger",
  "power_kw": 60.0,
  "access_type": "public",
  "base_price": 14.0,                 // INR per kWh
  "status": "active",
  "reliability_score": 0.94,          // 0.0 – 1.0 (Bayesian-smoothed)
  "parking_info": "B2 Near Elevator",
  "amenities": "WiFi, Food, Restroom",
  "latitude": 12.9716,
  "longitude": 77.5946,
  "created_at": "...",
  "updated_at": "...",
  "ports": [
    {
      "id": 1,
      "charger_id": 1,
      "connector_type": "CCS2",       // string: "CCS2"|"Type2"|"CHAdeMO"|"GB_T"|"Type1"
      "max_power_kw": 60.0,
      "status": "available",          // "available"|"occupied"|"offline"|"unknown"
      "created_at": "..."
    }
  ]
}
```

Connector Types

VALUE	DISPLAY NAME	TYPICAL POWER
CCS2	CCS2	50–150 kW (DC fast)
Type2	Type 2	7–22 kW (AC)
CHAdeMO	CHAdeMO	50–100 kW (DC fast)
GB_T	GB/T	30–120 kW (DC)
Type1	Type 1	7–19 kW (AC)

07 — Availability Window Management

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
GET	`/availability/port/{port_id}`	Public	—	`List<AvailabilityWindowResponse>`
POST	`/availability`	Owner	`AvailabilityWindowCreate`	`AvailabilityWindowResponse` (201)
PATCH	`/availability/{id}`	Owner	`AvailabilityWindowUpdate`	`AvailabilityWindowResponse`
DELETE	`/availability/{id}`	Owner	—	204 No Content

AvailabilityWindowCreate Schema

```
{
  "port_id": 1,                          // required
  "start_at": "2026-08-25T09:00:00Z",   // required
  "end_at": "2026-08-25T11:00:00Z",     // required, must be after start_at
  "source": "owner",                     // "owner"|"ai_recommendation"|"admin"
  "price_override": 18.0,                // optional, overrides charger base_price
  "status": "active",                    // "active"|"paused"|"cancelled"
  "is_recurring": false,
  "recurrence_rule": "RRULE:FREQ=WEEKLY;BYDAY=TU,TH"  // optional
}
```

08 — Booking System

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
POST	/bookings	Driver	BookingCreate	BookingResponse (201)
POST	/bookings/{id}/cancel	Driver	—	BookingResponse

Atomic Hold Lock: The backend uses Redis SET NX with a 600-second (10 min) TTL to prevent double-booking. If the lock can't be acquired, it returns 409 SLOT_UNAVAILABLE. On payment failure, the lock is released.

BookingCreate Schema

```
{
  "port_id": 1,                          // required
  "start_at": "2026-08-25T10:00:00Z",   // required, must be in future
  "end_at": "2026-08-25T11:00:00Z",     // required, must be after start_at
  "vehicle_id": 1,                       // optional
  "idempotency_key": "abc-123"          // optional, prevents duplicate submissions
}
```

Booking Status Machine

```
PENDING → HELD → PAYMENT_PENDING → CONFIRMED → CHECKED_IN → CHARGING → COMPLETED

Side paths: CANCELLED (user/admin), EXPIRED (hold timeout), FAILED (payment error), NO_SHOW (missed slot)
```

BookingResponse Schema

```
{
  "id": 1,
  "user_id": 2,
  "vehicle_id": 1,
  "port_id": 1,
  "start_at": "2026-08-25T10:00:00Z",
  "end_at": "2026-08-25T11:00:00Z",
  "status": "CONFIRMED",               // See status machine above
  "hold_expires_at": null,             // set when HELD, null after payment
  "quote_snapshot": {
    "price_per_kwh": 14.0,
    "estimated_kwh": 30.0,
    "estimated_total": 420.0
  },
  "idempotency_key": "abc-123",
  "created_at": "...",
  "updated_at": "..."
}
```

09 — Payment Integration (Razorpay)

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
POST	/payments/create-order	Driver	{ booking_id, amount, currency: "INR" }	PaymentResponse (201)
POST	/payments/webhook	Razorpay HMAC	Razorpay event payload	{ status: "ok" }

Payment Flow:
1. Frontend calls POST /payments/create-order → Backend creates Razorpay order + saves pending payment
2. Frontend opens Razorpay checkout sheet (mobile) or mock (web)
3. User completes payment inside Razorpay
4. Razorpay sends webhook to POST /payments/webhook
5. Backend verifies HMAC signature → Updates payment status → Transitions booking to CONFIRMED
6. Frontend polls or receives WebSocket notification of booking confirmation

PaymentResponse Schema

```
{
  "id": 1,
  "booking_id": 1,
  "amount": 420.0,
  "currency": "INR",
  "status": "completed",            // "pending"|"completed"|"failed"|"refunded"
  "provider_order_id": "order_abc123",  // Razorpay order ID
  "provider_payment_id": "pay_xyz789",  // Razorpay payment ID
  "verified_at": "2026-08-25T09:50:00Z",
  "created_at": "..."
}
```

10 — Charging Session Lifecycle

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
POST	/sessions/check-in	Driver	{ booking_id: int }	ChargingSessionResponse (201)
POST	/sessions/{id}/start	Driver	—	ChargingSessionResponse
POST	/sessions/{id}/complete	Driver	{ energy_kwh: float }	ChargingSessionResponse

Important: The backend calculates the final cost from `energy_kwh × charger.base_price`. The frontend sends only the energy consumed — never the price.

Session Lifecycle

Booking CONFIRMED → **POST /sessions/check-in** (booking_id) → CHECKED_IN → **POST /sessions/{id}/start** → CHARGING → **POST /sessions/{id}/complete** (energy_kwh) → COMPLETED

ChargingSessionResponse Schema

```
{
  "id": 1,
  "booking_id": 1,
  "check_in_at": "2026-08-25T10:02:00Z",
  "start_at": "2026-08-25T10:05:00Z",
  "end_at": "2026-08-25T10:35:00Z",
  "energy_kwh": 28.5,
  "final_amount": 399.0,            // calculated server-side
  "status": "completed",            // "checked_in"|"charging"|"completed"|"failed"
  "created_at": "..."
}
```

11 — Route Recommendations (ML-Powered)

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
POST	/recommendations	Driver	RecommendationRequest	RecommendationResponse
GET	/recommendations/business/{business_id}	Owner	—	Placeholder (ML under construction)

RecommendationRequest Schema

```
{
  "latitude": 18.5204,               // driver's current latitude
  "longitude": 73.8567,              // driver's current longitude
  "vehicle_id": 1,                    // driver's vehicle
  "current_soc": 0.45,               // current state of charge (0.0 – 1.0)
  "reserve_soc": 0.15,               // desired reserve (0.0 – 1.0)
  "target_soc": 0.80,                // target charge level (0.0 – 1.0)
  "radius_meters": 10000,            // search radius in meters
  "destination_lat": 18.5300,        // optional destination
  "destination_lng": 73.8800
}
```

How Recommendations Work (Backend Logic)

```
1. Fetch driver's vehicle (battery capacity, connector types, efficiency)
2. Spatial search: PostGIS nearby chargers within radius
3. Filter: active status + connector compatibility
4. Calculate: haversine distance, reachable with current SoC
5. For each reachable charger:
   ├── Best port power & estimated charge time
   ├── ML wait-time prediction (Model B: queue math)
   ├── Reliability score from charger.reliability_score
   └── Cost = energy_needed × base_price
6. Rank by weighted score (distance, wait, reliability, cost)
7. Return top 3 recommendations with reason codes
```

RecommendationResponse Schema

```
{
  "recommendations": [
    {
      "charger": { /* full ChargerResponse */ },
      "rank": 1,
      "detour_minutes": 6,
      "wait_minutes": 0,
      "charge_minutes": 27,
      "estimated_cost_inr": 205.0,
      "reliability_score": 0.94,
      "confidence": 0.91,
      "connector_compatible": true,
      "best_port_id": 1,
      "reason_codes": [
        "COMPATIBLE_CONNECTOR",
        "NO_QUEUE_EXPECTED",
        "HIGH_RELIABILITY",
        "SHORT_DETOUR"
      ]
    }
  ]
}
```

12 — Business Analytics & Intelligence

METHOD	ENDPOINT	AUTH	REQUEST	RESPONSE
GET	/analytics/businesses/{business_id}/recommendations	Owner	—	List<RecommendationResponse>

Business Intelligence Response

```
[
  {
    "charger_id": 1,
    "recommended_action": "Create Availability Window",
    "reason_code": "BUSINESS_OFF_PEAK",
    "expected_demand": 1.5,
    "suggested_discount_pct": 20,
    "confidence": 0.85
  },
  {
    "charger_id": 2,
    "recommended_action": "Enable Peak Pricing",
    "reason_code": "HIGH_DEMAND_LOW_SUPPLY",
    "expected_demand": 5.2,
    "suggested_discount_pct": 0,
    "confidence": 0.85
  }
]
```

13 — WebSocket Real-Time Updates

PROTOCOL	ENDPOINT	AUTH	DESCRIPTION
WS	/ws/{user_id}	User ID in path	Real-time session updates pushed from server

WebSocket Events

EVENT	DIRECTION	PAYLOAD
ping	Client → Server	{ "type": "ping" }
pong	Server → Client	{ "type": "pong" }
status_update	Server → Client	{ "type": "status_update", "data": { /* ChargingSession */ } }
session_completed	Server → Client	{ "type": "session_completed", "data": { /* session summary */ } }
error	Server → Client	{ "type": "error", "message": "..." }

> **Frontend reconnection:** Exponential backoff (1s → 2s → 4s → ... → 30s cap, max 10 attempts). Keepalive ping every 30s. Last known data stays visible during reconnection.

14 — ML Model Reference

Important: ML models are called internally by the backend via `MLAdapter`. They are NOT exposed as public API endpoints. The frontend never calls ML directly — it calls `/recommendations` or `/analytics` which invoke ML internally.

MODEL	PURPOSE	INPUT	OUTPUT	CALLED BY
Model A: Demand Forecast	Predict booking demand in next 60 minutes	charger_id, current hour, day of week	`{ expected_demand: float, confidence: float }`	`/analytics/.../recommendations`
Model B: Wait-time Predictor	Estimate queue wait for a specific port	charger_id, port_id, active sessions	`{ wait_minutes: float, congestion_level: "LOW"|"MEDIUM"|"HIGH", confidence: float }`	`/recommendations` (route planner)
Model C: Route Energy	Estimate energy needed for route segment	vehicle_id, route geometry, elevation	`{ energy_kwh: float, confidence: float }`	`/recommendations` (route planner)

ML Integration (Internal Endpoints)

METHOD	ENDPOINT	DESCRIPTION
POST	`/internal/ml/demand`	Demand forecast (internal, called by analytics service)
POST	`/internal/ml/availability`	Availability prediction (internal, called by recommendation service)

Current Implementation Status

Prototype phase: Models use deterministic heuristics (queue math, rolling averages, time-of-day patterns) as recommended by the playbook. The ML artifacts are trained on synthetic Pune data and packaged as `joblib` bundles. Full ML model integration (scikit-learn HGBR models) is ready for swap when backend integrates the `voltez-ml` package.

15 — Complete Data Model Reference

All Backend Enums

MODEL	FIELD	ALLOWED VALUES
User	role	"DRIVER", "OWNER", "ADMIN"
User	verification_status	"unverified", "verified"
Vehicle	connector_types	["CCS2", "Type2", "CHAdeMO", "GB_T", "Type1"]
Business	verification_status	"unverified", "PENDING", "verified"
Business	category	"mall", "office", "apartment"
Charger	status	"active", "paused", "inactive"
Charger	access_type	"public", "private", "restricted"
ChargerPort	status	"available", "occupied", "offline", "unknown"
ChargerPort	connector_type	"CCS2", "Type2", "CHAdeMO", "GB_T", "Type1"
Booking	status	"PENDING", "HELD", "PAYMENT_PENDING", "CONFIRMED", "CANCELLED", "EXPIRED", "FAILED", "NO_SHOW", "CHECKED_IN", "CHARGING", "COMPLETED"
ChargingSession	status	"checked_in", "charging", "completed", "failed"
Payment	status	"pending", "completed", "failed", "refunded"
AvailabilityWindow	status	"active", "paused", "cancelled"
AvailabilityWindow	source	"owner", "ai_recommendation", "admin"

Frontend Dart Models

> All Dart models are in `lib/shared/models/models.dart`. Each model has `fromJson` factory constructors that handle both backend format and legacy frontend format. The `BookingStatus` enum uses UPPER_CASE names intentionally to match backend JSON strings directly.

16 — Error Handling

Standard Backend Error Response

```
{
  "detail": "Error message here",         // or "code" + "message"
  "code": "SLOT_UNAVAILABLE",             // machine-readable error code
  "request_id": "req_abc123"              // for debugging
}
```

HTTP Status Codes Used

STATUS	MEANING	WHEN
200	OK	Successful GET/PATCH
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Invalid Razorpay webhook signature
401	Unauthorized	Missing/invalid JWT, expired refresh token
403	Forbidden	Role doesn't match (e.g., DRIVER accessing OWNER endpoint)
404	Not Found	Resource doesn't exist or user doesn't own it
409	Conflict	SLOT_UNAVAILABLE (Redis lock failed, double-booking)
422	Validation Error	Pydantic validation failed (bad field types, missing required fields)
502	Bad Gateway	Razorpay API is unreachable

Frontend Error Handling (Flutter)

> The `_ErrorInterceptor` in `api_service.dart` catches all HTTP errors and parses them into a typed `ApiError` object with `code`, `message`, `statusCode`, and `fieldErrors`. Providers handle errors by setting `_error` state and showing retry UI.

17 — Frontend Integration Status

FRONTEND FEATURE	BACKEND ENDPOINT	STATUS	NOTES
Register	`POST /auth/register`	✅ Live	
Login	`POST /auth/login`	✅ Live	
Token Refresh	`POST /auth/refresh`	✅ Live	Auto-refresh on 401
Get Profile	`GET /users/me`	✅ Live	
Vehicle CRUD	`/vehicles/*`	✅ Live	
Nearby Chargers	`GET /chargers/nearby`	✅ Live	PostGIS spatial query
Charger Details	`GET /chargers/{id}`	✅ Live	
Create Booking	`POST /bookings`	✅ Live	Redis atomic lock
Cancel Booking	`POST /bookings/{id}/cancel`	✅ Live	
Payment Order	`POST /payments/create-order`	✅ Live	Razorpay integration
Payment Webhook	`POST /payments/webhook`	✅ Live	HMAC verification
Check-in	`POST /sessions/check-in`	✅ Live	
Start Charging	`POST /sessions/{id}/start`	✅ Live	
Complete Session	`POST /sessions/{id}/complete`	✅ Live	
Route Recommendations	`POST /recommendations`	✅ Live	ML-powered ranking
Business Recommendations	`GET /analytics/.../recommendations`	✅ Live	Demand forecast → pricing advice
Business CRUD	`/businesses/*`	✅ Live	
Availability CRUD	`/availability/*`	✅ Live	
WebSocket Updates	`ws:///ws/{user_id}`	✅ Live	Session telemetry
Report Charger Issue	`POST /chargers/{id}/report-issue`	✅ Live	Penalizes reliability score

> **All 20 frontend features are connected to real backend endpoints.** Mock adapters remain in place for testing without a live backend — swap `Mock*` for `Live*` at the DI root in `main.dart`.

VoltEZ — Smart EV Charging Platform
API Reference v1.0 | Generated August 2026
Backend: FastAPI + PostgreSQL/PostGIS + Redis | Frontend: Flutter (Dart) | ML: scikit-learn + joblib
⚡ Built for Hackathon Demo